Practical : 1	**Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort**
Date : 01/07/2026	

Aim:

To implement and analyze different sorting algorithms such as **Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort**, each as a separate program.

Objective :-

To implement five major sorting algorithms (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quicksort) in C++ using arrays, and to evaluate their combined time and space complexity characteristics.

SORTING ALGORITHMS

1. Bubble Sort

1. Start.
2. Read the array and size n.
3. Compare adjacent elements.
4. If the first element is greater than the next element, swap them.
5. Repeat the comparison for the remaining elements.
6. After each pass, the largest element moves to the end.
7. Repeat until the array is sorted.
8. Stop.

2. Selection Sort

1. Start.
2. Read the array and size n.
3. Select the first unsorted position.
4. Find the minimum element in the remaining array.
5. Swap the minimum element with the current position.
6. Move to the next position.
7. Repeat until the array is sorted.
8. Stop.

3. Insertion Sort

1. Start.
2. Read the array and size n.
3. Select the next element as key.
4. Compare the key with elements in the sorted portion.
5. Shift larger elements one position to the right.
6. Insert the key in its correct position.
7. Repeat for all elements.
8. Stop.

4. Merge Sort

1. Start.
2. Divide the array into two halves.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.
6. Repeat until the complete array is sorted.
7. Stop.

5. Quick Sort

1. Start.
2. Select the last element as the pivot.
3. Partition the array around the pivot.
4. Place smaller elements before the pivot.
5. Place larger elements after the pivot.
6. Recursively sort the left and right portions.
7. Stop.

Program :-

Program 1: Bubble Sort (practical1_bubble.cpp)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace chrono;
// Bubble Sort
void bubbleSort(vector<int>& arr)
{
    int n = arr.size();
    for(int i = 0; i < n-1; i++)
    {
        for(int j = 0; j < n-i-1; j++)
        {
            if(arr[j] > arr[j+1])
            {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
int main()
{
    srand(time(0));
    int sizes[] = {100, 500, 1000, 5000, 10000};
    cout << "Bubble Sort Performance Analysis\n\n";
    for(int n : sizes)
    {
        vector<int> arr(n);
        // Generate random numbers
        for(int i = 0; i < n; i++)
        {
            arr[i] = rand() % 1000;
        }
        vector<int> temp = arr;
        auto start = high_resolution_clock::now();
        bubbleSort(temp);
        auto stop = high_resolution_clock::now();
        long long sortTime =
            duration_cast<microseconds>(stop-start).count();
        cout << "Number of Elements = " << n << endl;
        cout << "Bubble Sort Time : "
             << sortTime << " microseconds" << endl;
        cout << "----------------------------------------\n";
    }
    return 0;
}
```

Output:

```
Bubble Sort Performance Analysis

Number of Elements = 100
Bubble Sort Time : 41 microseconds
----------------------------------------
Number of Elements = 500
Bubble Sort Time : 982 microseconds
----------------------------------------
Number of Elements = 1000
Bubble Sort Time : 4154 microseconds
----------------------------------------
Number of Elements = 5000
Bubble Sort Time : 111739 microseconds
----------------------------------------
Number of Elements = 10000
Bubble Sort Time : 449486 microseconds
----------------------------------------
```

Program 2: Selection Sort (practical1_selection.cpp)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace chrono;
// Selection Sort
void selectionSort(vector<int>& arr)
{
    int n = arr.size();
    for(int i = 0; i < n-1; i++)
    {
        int min = i;
        for(int j = i+1; j < n; j++)
        {
            if(arr[j] < arr[min])
            {
                min = j;
            }
        }
        swap(arr[i], arr[min]);
    }
}
int main()
{
    srand(time(0));
    int sizes[] = {100, 500, 1000, 5000, 10000};
    cout << "Selection Sort Performance Analysis\n\n";
    for(int n : sizes)
    {
        vector<int> arr(n);
        // Generate random numbers
        for(int i = 0; i < n; i++)
```

```
        {
            arr[i] = rand() % 1000;
        }
        vector<int> temp = arr;
        auto start = high_resolution_clock::now();
        selectionSort(temp);
        auto stop = high_resolution_clock::now();
        long long sortTime =
            duration_cast<microseconds>(stop-start).count();
        cout << "Number of Elements = " << n << endl;
        cout << "Selection Sort Time : "
             << sortTime << " microseconds" << endl;
        cout << "----------------------------------------\n";
    }
    return 0;
}
```

Output:

```
Selection Sort Performance Analysis

Number of Elements = 100
Selection Sort Time : 83 microseconds
----------------------------------------
Number of Elements = 500
Selection Sort Time : 350 microseconds
----------------------------------------
Number of Elements = 1000
Selection Sort Time : 1422 microseconds
----------------------------------------
Number of Elements = 5000
Selection Sort Time : 32301 microseconds
----------------------------------------
Number of Elements = 10000
Selection Sort Time : 144340 microseconds
----------------------------------------
```

Program 3: Insertion Sort (practical1_insertion.cpp)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace chrono;
// Insertion Sort
void insertionSort(vector<int>& arr)
{
    int n = arr.size();
    for(int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i-1;
        while(j >= 0 && arr[j] > key)
        {
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
int main()
{
    srand(time(0));
    int sizes[] = {100, 500, 1000, 5000, 10000};
    cout << "Insertion Sort Performance Analysis\n\n";
    for(int n : sizes)
    {
        vector<int> arr(n);
        // Generate random numbers
        for(int i = 0; i < n; i++)
        {
            arr[i] = rand() % 1000;
        }
        vector<int> temp = arr;
        auto start = high_resolution_clock::now();
        insertionSort(temp);
        auto stop = high_resolution_clock::now();
        long long sortTime =
            duration_cast<microseconds>(stop-start).count();
        cout << "Number of Elements = " << n << endl;
        cout << "Insertion Sort Time : "
             << sortTime << " microseconds" << endl;
        cout << "----------------------------------------\n";
    }
    return 0;
}
```

Output:

```
Insertion Sort Performance Analysis

Number of Elements = 100
Insertion Sort Time : 12 microseconds
----------------------------------------
Number of Elements = 500
Insertion Sort Time : 275 microseconds
----------------------------------------
Number of Elements = 1000
Insertion Sort Time : 1061 microseconds
----------------------------------------
Number of Elements = 5000
Insertion Sort Time : 23222 microseconds
----------------------------------------
Number of Elements = 10000
Insertion Sort Time : 93631 microseconds
----------------------------------------
```

Program 4: Merge Sort (practical1_merge.cpp)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace chrono;
// Merge Sort
void merge(vector<int>& arr, int l, int m, int r)
{
    int n1 = m-l+1;
    int n2 = r-m;
    vector<int> left(n1);
    vector<int> right(n2);
    for(int i = 0; i < n1; i++)
        left[i] = arr[l+i];
    for(int j = 0; j < n2; j++)
        right[j] = arr[m+1+j];
    int i = 0;
    int j = 0;
    int k = l;
    while(i < n1 && j < n2)
    {
        if(left[i] <= right[j])
        {
            arr[k] = left[i];
            i++;
        }
        else
        {
            arr[k] = right[j];
            j++;
        }
        k++;
```

```
    }
    while(i < n1)
    {
        arr[k] = left[i];
        i++;
        k++;
    }
    while(j < n2)
    {
        arr[k] = right[j];
        j++;
        k++;
    }
}
void mergeSort(vector<int>& arr, int l, int r)
{
    if(l < r)
    {
        int m = (l+r)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}
int main()
{
    srand(time(0));
    int sizes[] = {100, 500, 1000, 5000, 10000};
    cout << "Merge Sort Performance Analysis\n\n";
    for(int n : sizes)
    {
        vector<int> arr(n);
        // Generate random numbers
        for(int i = 0; i < n; i++)
        {
            arr[i] = rand() % 1000;
        }
        vector<int> temp = arr;
        auto start = high_resolution_clock::now();
        mergeSort(temp, 0, n-1);
        auto stop = high_resolution_clock::now();
        long long sortTime =
            duration_cast<microseconds>(stop-start).count();
        cout << "Number of Elements = " << n << endl;
        cout << "Merge Sort Time : "
             << sortTime << " microseconds" << endl;
        cout << "----------------------------------------\n";
    }
    return 0;
}
```

Output:

```
Merge Sort Performance Analysis

Number of Elements = 100
Merge Sort Time : 27 microseconds
----------------------------------------
Number of Elements = 500
Merge Sort Time : 132 microseconds
----------------------------------------
Number of Elements = 1000
Merge Sort Time : 288 microseconds
----------------------------------------
Number of Elements = 5000
Merge Sort Time : 1631 microseconds
----------------------------------------
Number of Elements = 10000
Merge Sort Time : 3374 microseconds
----------------------------------------
```

Program 5: Quick Sort (practical1_quick.cpp)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace chrono;
// Quick Sort
int partition(vector<int>& arr, int low, int high)
{
    int pivot = arr[high];
    int i = low-1;
    for(int j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}
void quickSort(vector<int>& arr, int low, int high)
{
    if(low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p-1);
        quickSort(arr, p+1, high);
    }
}
int main()
{
```

```
    srand(time(0));
    int sizes[] = {100, 500, 1000, 5000, 10000};
    cout << "Quick Sort Performance Analysis\n\n";
    for(int n : sizes)
    {
        vector<int> arr(n);
        // Generate random numbers
        for(int i = 0; i < n; i++)
        {
            arr[i] = rand() % 1000;
        }
        vector<int> temp = arr;
        auto start = high_resolution_clock::now();
        quickSort(temp, 0, n-1);
        auto stop = high_resolution_clock::now();
        long long sortTime =
            duration_cast<microseconds>(stop-start).count();
        cout << "Number of Elements = " << n << endl;
        cout << "Quick Sort Time : "
             << sortTime << " microseconds" << endl;
        cout << "----------------------------------------\n";
    }
    return 0;
}
```

Output:

```
Quick Sort Performance Analysis

Number of Elements = 100
Quick Sort Time : 8 microseconds
----------------------------------------
Number of Elements = 500
Quick Sort Time : 51 microseconds
----------------------------------------
Number of Elements = 1000
Quick Sort Time : 109 microseconds
----------------------------------------
Number of Elements = 5000
Quick Sort Time : 688 microseconds
----------------------------------------
Number of Elements = 10000
Quick Sort Time : 1446 microseconds
----------------------------------------
```

Analysis:-

The algorithms tested can be naturally grouped into two categories based on their design patterns:
Quadratic-time basic sorts O(n^2) and **Logarithmic-time advanced divide-and-conquer sorts** O(n log n)

Time Complexity Summary

Bubble, Selection, and Insertion Sort utilize nested loop structures. In average and worst-case setups, they complete roughly **n^2** computational comparison loops, giving them an **O(n^2)** quadratic rate of growth. Insertion and optimized Bubble Sort perform exceptionally well **O(n)** if the dataset is already mostly sorted.

Merge Sort and Quicksort split the data into smaller fractions continuously. This divide-and-conquer strategy dramatically cuts down operations to an **O(n log n)** average rate.

Quicksort can degrade to **O(n^2)** if the pivot choices consistently isolate single elements instead of balanced splits.

Sorting Algorithms	Time Complexity			Space Complexity
	Best Case	Average Case	Worst Case	Worst Case
Bubble Sort	Ω(N)	Θ(N^2)	O(N^2)	O(1)
Selection Sort	Ω(N^2)	Θ(N^2)	O(N^2)	O(1)
Insertion Sort	Ω(N)	Θ(N^2)	O(N^2)	O(1)
Quick Sort	Ω(N log N)	Θ(N log N)	O(N^2)	O(N)
Merge Sort	Ω(N log N)	Θ(N log N)	O(N log N)	O(N)

Space Complexity Summary

a) Bubble, Selection, and Insertion Sort require **O(1)** space because they arrange elements completely in-place without generating extra lists.

b) Merge Sort requires a linear auxiliary footprint of **O(n)** memory to construct temporary arrays during active array blending operations.

c) Quicksort uses an execution stack footprint of **O(log n)** on average to keep track of recursive partition boundaries.

Conclusion :-

This practical highlights the direct real-world trade-off between algorithmic simplicity and scalability:

For Small Datasets: Simple algorithms like **Insertion Sort** and **Bubble Sort** are optimal because they require negligible implementation complexity and execute completely in place (O(1) auxiliary space).

For Large Datasets: Divide-and-conquer algorithms like **Merge Sort** and **Quicksort** are vastly superior because their O(n log n) runtime allows them to process massive files cleanly. **Merge Sort** guarantees reliable stability but requires a higher memory profile (O(n)), while **Quicksort** offers highly rapid in-place processing under typical runtime conditions.